Riassunto hybrid methods

Hybrid Methods: Plug and Play

Model based approach

Questo metodo si basa sulla risoluzione di un problema di ottimizzazione in cui si cerca di bilanciare la fedeltà ai dati fisici e un regolarizzatore scelto manualmente.

- **Vantaggi (+):**
  - **Fisica e Statistica:** Incorpora esplicitamente il modello fisico dell'acquisizione e le conoscenze a priori (priors).
  - **Sperimentabilità e Spiegabilità:** I risultati sono giustificati da formulazioni matematiche trasparenti; ogni termine dell'equazione ha un significato fisico chiaro.
- **Svantaggi (-):**
  - **Scelte Arbitrarie:** La scelta della funzione di regolarizzazione e del relativo parametro spesso richiede lunghi tentativi manuali (*tuning*).
  - **Lentezza:** Essendo metodi iterativi, richiedono tempi di calcolo molto lunghi, rendendoli poco pratici per applicazioni in tempo reale.

From model based to data driven

Il passaggio verso il Deep Learning è guidato dalla necessità di superare i limiti dell'approccio classico:

1. **Velocità ed Efficienza:** Mentre l'MBIR è iterativo e costoso, una rete neurale, una volta addestrata, fornisce la ricostruzione quasi istantaneamente (**real-time**).
2. **Apprendimento dei Priors:** Invece di "disegnare" a mano un regolarizzatore (es. Total Variation), le reti imparano automaticamente le caratteristiche complesse delle immagini dai dataset.
3. **Ottimizzazione End-to-End:** La rete mappa direttamente i dati grezzi alla ricostruzione finale, ottimizzando l'intero processo in un unico passaggio, senza dover risolvere sequenzialmente i singoli step.

Data driven methods

In questo paradigma, il "sapere" è estratto da grandi quantità di esempi.

- **Vantaggi (+):**
  - **Esecuzione Ultra-rapida:** Ideale per contesti clinici o industriali dove il tempo è critico.
  - **Qualità Superiore:** Sfruttando informazioni provenienti da migliaia di dati, le reti possono ricostruire dettagli che i modelli classici non riescono a catturare.
  - **Robustezza al Rumore:** Possono essere addestrate per gestire pattern di rumore molto diversificati, migliorando la stabilità in condizioni difficili.
- **Svantaggi (-):**

- **Dipendenza dai Dati:** Richiedono dataset enormi e di alta qualità. Se i dati di addestramento sono scarsi o errati, il modello sarà mediocre.
- **Problemi di Generalizzazione:** Le reti possono diventare instabili o fallire se incontrano dati troppo diversi da quelli visti durante l'addestramento (*out-of-distribution*).
- **"Black Box":** A differenza dell'MBIR, è spesso difficile capire *perché* una rete abbia prodotto un certo risultato, mancando di una formulazione matematica esplicita per la ricostruzione.

The Plug and Play approach

Gli approcci ibridi permettono di mantenere il **data consistency** (ovvero la certezza che il risultato sia coerente con le misurazioni fisiche effettuate) sfruttando al contempo la capacità delle reti di modellare informazioni a priori estremamente complesse.

Il framework **Plug-and-Play (PnP)** è un metodo avanzato che permette di incorporare modelli a priori (*prior*) complessi all'interno di problemi inversi.

- L'obiettivo fondamentale è fondere l'approccio **model-based** (basato sulla fisica) con modelli basati sull'azione, come le **Reti Neurali Profonde (DNN)**, progettate per generare un output a partire da un input.
- Nel framework PnP, il modello a priori tradizionale viene sostituito da una classe generale di **operatori di denoising** (riduzione del rumore), che possono essere implementati in modo estremamente efficiente tramite le DNN.

L'idea centrale è quella di **disaccoppiare** il modello forward (la fisica della misurazione) dal *prior* (il modello dell'immagine):

$$\min_x \|Ax - y^\delta\|_2^2 + \lambda \mathcal{R}(x)$$

dove la parte dei minimi quadrati rappresenta il termine di fedeltà ai dati, $\mathcal{R}(\boldsymbol{x})$ è il termine di regolarizzazione e λ è un parametro di bilanciamento (*trade-off*).

> Il termine di fedeltà garantisce che la soluzione sia coerente con il processo di degradazione, mentre il termine di regolarizzazione impone le proprietà desiderate all'output. In generale, i metodi per risolvere questo problema di minimizzazione possono essere divisi in due categorie principali: i metodi di ottimizzazione **model-based** e i metodi di **apprendimento discriminativo** (*data-driven*).
>
> I metodi di ottimizzazione **model-based** mirano a risolvere direttamente il problema di minimizzazione con algoritmi che solitamente comportano un'inferenza iterativa dispendiosa in termini di tempo. Al contrario, i metodi di **apprendimento discriminativo** cercano di apprendere i parametri a priori e un'inferenza compatta attraverso l'ottimizzazione di una funzione di perdita (*loss function*) su un set di addestramento contenente coppie di immagini degradate e pulite.
>
> Si può notare che una differenza evidente tra i due approcci è che il primo è flessibile nel gestire vari compiti di ricostruzione (IR) specificando la matrice di degradazione H, mentre il secondo deve utilizzare dati di addestramento con specifiche matrici di degradazione per apprendere il modello. Di conseguenza, a differenza dei metodi model-based, i metodi di apprendimento discriminativo sono solitamente limitati a compiti specializzati.
>
> Tuttavia, a scapito della flessibilità, i metodi discriminativi non solo godono di un'elevata velocità in fase di test, ma tendono anche a fornire prestazioni promettenti grazie all'ottimizzazione congiunta e all'addestramento *end-to-end*. Al contrario, i metodi model-based richiedono solitamente molto tempo e l'uso di *priors* sofisticati per ottenere buone prestazioni.

The Half Quadratic Splitting (HQS) algorithm

Consideriamo il modello precedente:

$$\min_x \|Ax - y^\delta\|_2^2 + \lambda \mathcal{R}(x)$$

Introducendo una variabile ausiliaria z, il metodo **Half Quadratic Splitting (HQS)** tenta di risolvere il problema vincolato:

$$\min_{x,z} \frac{1}{2}\|Ax - y^\delta\|_2^2 + \lambda \mathcal{R}(z), \quad \text{s.t. } z = x$$

Questo è equivalente a risolvere:

$$\min_{x,z} \frac{1}{2}\|Ax - y^\delta\|_2^2 + \lambda \mathcal{R}(z) + \frac{\mu}{2}\|z - x\|_2^2$$

dove μ è un parametro di penalità. Il problema di minimizzazione viene risolto mediante il seguente schema di scissione (*splitting*):

$$x_{k+1} = \arg\min_x \|Ax - y^\delta\|_2^2 + \mu\|x - z_k\|_2^2$$
$$z_{k+1} = \arg\min_z \frac{\mu}{2}\|z - x_{k+1}\|_2^2 + \lambda \mathcal{R}(z)$$

In questo modo, il termine di fedeltà e il termine di regolarizzazione vengono **disaccoppiati** in due equazioni differenti.

- **La prima equazione** è un problema di regolarizzazione di Tikhonov che può essere risolto con un metodo adeguato (come il gradiente coniugato).
- **La seconda equazione** rappresenta il **denoising** dell'immagine x_{k+1}. Qui è possibile "inserire" (*plug-in*) un **denoiser** qualsiasi.

Questa struttura è ciò che rende possibile l'approccio Plug-and-Play: se sostituiamo la soluzione formale della seconda equazione con il passaggio di x_{k+1} attraverso una rete neurale denoiser D_σ, abbiamo effettivamente integrato il deep learning in un framework di ottimizzazione matematicamente solido.

La seconda equazione rappresenta il **denoising di Dx**, ovvero dell'immagine del gradiente di x. Di conseguenza, è possibile inserire una rete neurale che agisca come denoiser sui gradienti. Si può utilizzare una **ResUNet**, ad esempio con un livello finale che estrae i gradienti dall'immagine di output, oppure addestrarla direttamente sui gradienti dell'immagine partendo dai sinogrammi rumorosi.

Hybrid Methods: Unrolling Algorithms

In termini semplici, l'**unrolling** consiste nel trasformare un tradizionale algoritmo di ottimizzazione iterativo in una rete neurale profonda, dove ogni iterazione corrisponde a un livello (*layer*) della rete.

L'unrolling costruisce una rete supervisionata addestrabile che imita il processo di ottimizzazione ma è:

- molto più **veloce**,
- **adattata** alla specifica distribuzione dei dati,
- capace di apprendere implicitamente **prior più potenti**.

Esempio: Srotolamento della discesa del gradiente per risolvere $\min_x f(x) = \|Ax - y^\delta\|_2^2 + \lambda \mathcal{R}(x)$ e apprendere il parametro di regolarizzazione λ. Questo è il caso più semplice possibile, in cui la rete deve apprendere un singolo parametro.

Ad ogni iterazione k, la discesa del gradiente aggiorna la soluzione come:

$$x_{k+1} = x_k - \eta_k \nabla f(x_k)$$

Nell'unrolling, reinterpretiamo ogni iterazione della discesa del gradiente come un livello in una rete neurale profonda. Invece di scegliere manualmente i parametri, come il parametro di regolarizzazione o la dimensione del passo η_k , li apprendiamo (insieme eventualmente ad altri parametri) direttamente dai dati.

Per costruire una rete di questo tipo, si seguono tre passaggi fondamentali:

1. **Scelta delle iterazioni:** Si fissa un numero prestabilito di iterazioni, ad esempio T.
2. **Costruzione della sequenza:** Si crea una sequenza di T aggiornamenti, dove ogni aggiornamento corrisponde a un livello della rete:
   - **Forward pass:** Simula la discesa del gradiente (o un altro algoritmo di ottimizzazione) attraverso i T passaggi.
   - **Backward pass:** Calcola i gradienti della funzione di perdita rispetto ai parametri (come le dimensioni dei passi o le modifiche apprese).
3. **Addestramento supervisionato:** Si addestrano i parametri (step size o operatori appresi) utilizzando coppie di dati:
   - **Input:** Misurazioni corrotte o incomplete (y^{δ}).
   - **Target:** Immagini pulite o segnali reali (*ground truth* x_{GT}).

Algoritmi ancora più avanzati apprendono direttamente la funzione di regolarizzazione $\mathcal{R}(x)$, ipotizzando che essa dipenda da un insieme di parametri θ appresi dalla rete neurale, $\mathcal{R}_{\theta}(x)$

Vantaggi degli unrolling algorithms:

1. **Architettura Interpretabile**
2. **Migliore Generalizzazione**
3. **Efficienza**
4. **Regolarizzazione Apprendibile**
5. **Flessibilità**
6. **Minor numero di parametri**

Svantaggi degli unrolling algorithms:

1. **Numero limitato di iterazioni**
2. **Profondità fissa (Fixed Depth)**
3. **Overfitting alla distribuzione di addestramento**
4. **Perdita di garanzie teoriche**

Other Hybrid Methods

RISING for sparse-view or limited angle tomography

> **Evangelista, Davide, Elena Morotti e Elena Loli Piccolomini**. *"RISING: A new framework for model-based few-view CT image reconstruction with deep learning."* Computerized Medical Imaging and Graphics 103 (2023): 102156.

Nella TAC a vista sparsa (*sparse-view*) o ad angolo limitato (*limited angle*), una ricostruzione grezza ma veloce viene ottenuta tramite la **Retroproiezione Filtrata** (Filtered Back-Projection, FBP), seguita da una rete neurale, tipicamente una **U-Net**, addestrata sulle immagini "ground truth" per rimuovere rumore e artefatti.

Poiché è piuttosto difficile disporre di dataset con "ground truth" (verità di base) nella CT (e nell'imaging medico in generale), possiamo creare il nostro dataset target ricostruendo il sinogramma tramite molte iterazioni di un metodo **MBIR** con parametri adeguati. Questo porta al cosiddetto framework **"RISING".**

La differenza principale rispetto al paradigma precedente, chiamato **LPP** (*Learned Post-Processing*), risiede nei dati target utilizzati per l'addestramento della rete: nel primo caso si utilizzano immagini *ground truth* reali, mentre nel secondo caso si utilizzano immagini ricostruite tramite un metodo **MBIR**.

Questa evoluzione ha portato allo sviluppo di tecniche ancora più avanzate, come descritto nell'articolo:

> **Elena Loli Piccolomini, Davide Evangelista e Elena Morotti.** *"Deep Guess acceleration for explainable image reconstruction in sparse-view CT."* arXiv preprint arXiv:2412.01703 (2024).

Initial Guess (or Deep Guess)

I metodi di regolarizzazione non convessa, come la **TpV** (*Total p-Variation*), sono estremamente efficaci nel rimuovere rumore e artefatti preservando bordi e strutture. Tuttavia, presentano un grosso limite: essendo non convessi, possono avere molteplici **minimi locali**.

Se disponiamo di una buona approssimazione della soluzione, possiamo usarla come **stima iniziale** (*initial guess*) per l'algoritmo di ottimizzazione. Questo aumenta drasticamente la probabilità di convergere verso il minimo globale o, quanto meno, verso un minimo locale di alta qualità vicino a quello globale.

L'idea alla base di **Deep Guess** è la seguente:

- Utilizzare una rete neurale convoluzionale, nello specifico una **ResUNet**.
- Addestrare la rete su immagini *ground truth*
- La rete viene utilizzata per **stimare la soluzione iniziale**, fornendo all'algoritmo non convesso un punto di partenza già molto vicino alla realtà.

Si tratta di un framework basato su due fasi principali:

1. **Calcolo della Deep Guess (DG):** Si calcola un'approssimazione della soluzione utilizzando una rete neurale (preferibilmente un'architettura semplice e non troppo pesante dal punto di vista computazionale). L'output di questa fase è definito, appunto, **Deep Guess**.
2. La seconda fase raffina la qualità dell'immagine calcolata dalla **Deep Guess (DG)** applicando ad essa alcune iterazioni di un algoritmo di ottimizzazione iterativo che risolve il problema di minimizzazione non convessa:

   $$\min_x \|Ax - y^\delta\| + \lambda TpV(x)$$

   Per quest'ultimo passaggio possiamo utilizzare l'algoritmo di **Chambolle-Pock**.

Questo garantisce la **spiegabilità** (*explainability*) del risultato ottenuto, in quanto punto di convergenza (o punto prossimo ad esso) di un algoritmo di ottimizzazione ben definito matematicamente.

Spatially adaptive regularization: weighted TV

Approccio **MBIR** con parametro di regolarizzazione variante nello spazio, ovvero la **Total Variation pesata** (weighted Total Variation)

$$\min_x \|Ax - y^\delta\|_2^2 + \lambda TV_w(x)$$

dove

$$TV_w(x) = \sum_{i=1}^{N} w_i \sqrt{(D_h x)_i^2 + (D_v x)_i^2}$$

e $w = (w_1, w_2, \dots w_N)$ sono i pesi. Questi pesi permettono l'adattamento dello sforzo di regolarizzazione alle caratteristiche della specifica immagine.

Il **ricostruttore** Ψ calcola l'immagine $\tilde{x}$. In linea di principio, può essere un qualsiasi metodo di ricostruzione, ma possiamo inserire (**plug-in**) una rete neurale che calcoli un'approssimazione $\tilde{x}$ della *ground truth* x_{GT} con grande accuratezza. Possiamo utilizzare una **Residual U-Net** come rete neurale, oppure un **Transformer**.